

Cricket Answer-Router: when does each RAG flavor win?

A cricket QA agent that routes between text-RAG, structured stats-SQL, and (phase 2) GraphRAG, with a measured comparison of where each retrieval strategy succeeds and fails.

Group: [G____, <Group Name>] | **Primary framework:** Pydantic AI | **Domain:** Cricket (Intl men's Test/ODI/T20I + IPL)

PROBLEM STATEMENT

Cricket knowledge lives in two incompatible shapes. **Prose** holds narrative: why the 2019 World Cup final was controversial, what made Dravid's technique distinctive. **Structured records** hold facts: who scored the most runs in IPL 2016, the highest ODI chase. A single retrieval strategy can only serve one shape well. Vector RAG over prose cannot compute a record; text-to-SQL cannot explain why a match mattered. Prose also goes **stale** the moment a record breaks, so a retrieval-only system answers record questions confidently and wrongly.

We build a cricket QA agent that routes a question to the right retrieval strategy, and we measure, on a held-out gold set, which strategy wins and fails on which question type. **Research question:** does an agentic router beat every single RAG flavor, and where does each flavor break? **Success** is a routing table showing no single arm dominates and the router captures best-of-all.

DATA INGESTION PIPELINE

- Cricsheet** (open ball-by-ball, verified: 885 Tests / 2548 ODIs / 3444 T20Is / 1243 IPL) → streamed into **SQLite**. Backs the stats arm and serves as the **computed oracle** for stats questions.
- Wikipedia** cricket articles → cleaned, chunked, embedded (sentence-transformers) into **Qdrant**. Backs the text-RAG arm.
- Sports Stack Exchange** cricket tag (verified: 839 Qs, **459 with accepted answers**) → SQLite. Accepted answers are the **narrative oracle**.
- Provenance carried on every row: source URL, author id, creation date (CC-BY-SA attribution).
- Frozen, seeded holdout** split out before any indexing, so the eval set never leaks into retrieval.
- Staleness is engineered honestly: the Wikipedia snapshot is frozen ~1–2 yrs behind the Cricsheet cutoff, so broken records exist only in the structured source. Documented as a deliberate rig.

RETRIEVAL DESIGN & ORCHESTRATION

Every arm implements one interface, `answer(q) → (text, sources)`, so the harness treats them uniformly. We build the **simple baselines first**, measure them, and only then add the router.

- Arm 1 — Text-RAG (baseline):** embed → top-k Qdrant retrieve with metadata filters (format/season/team) → Claude answers from context.
- Arm 2 — Stats-SQL (baseline):** Claude text-to-SQL over the fixed Cricsheet schema → execute → format.
- Arm 3 — Router agent (the ReAct loop):** Pydantic AI on a current Claude model, with Arms 1–2 plus a frozen `current_records` tool exposed as tools. The agent decides per question which tool(s) to call and can chain. This is the orchestration under study.
- Arm 4 — GraphRAG** Phase 2 text-derived community summaries (Neo4j) for global/thematic questions that top-k RAG structurally cannot answer.

COMPARATIVE EVALUATION & FAILURE ANALYSIS

Gold set is found and computed, never hand-authored: **stats** questions are templated from Cricsheet with SQL-computed answers (objective exact/numeric match); **narrative** questions are sampled from Sports SE accepted answers, scored by **LLM-as-judge** (reported as the softer metric). We report a per-arm accuracy table by question type plus per-question failure logs.

Question type	Text-RAG	Stats-SQL	GraphRAG	Router (target)
Narrative / local	high	~0	low	high
Numeric / record	stale, low	high	~0	high
Global / thematic P2	low	~0	high	high

Hypothesis, not results. Actual numbers are produced by the harness on the frozen holdout. A router that fails to beat the arms is itself a documented finding (routing errors, tool-selection overhead) in the failure analysis.

DECISION REASONING

- Corpus forces technique.** Cricket is chosen because its native split (prose vs records vs graph) makes each retrieval flavor *necessary* rather than bolted on, so the comparison is real.
- Baseline before advanced.** Two single-strategy baselines are measured before the router, so the router's added value is quantified, not assumed.
- Pydantic AI** over LangGraph/smolagents: type-safe tools and structured outputs, thin and explicit about the loop, native Claude support. Keeps the ReAct mechanics visible for a learning project.
- Scope discipline.** One narrow question, phased delivery: 3-arm MVP first, GraphRAG second.